



Pothole Eye

User Manual

Real-time pothole detection and alerting system

Deployment & Operations Guide

May 2026

Mahade Mishuk
mdmis001@goucher.edu
Computer Science Department, Goucher College, MD
Class of 2026

01 Overview

Pothole Eye is an AI-powered pothole detection system that runs on GPU-accelerated cloud infrastructure (RunPod) and is accessed through a web browser on your local machine. The system uses a YOLOv8 computer-vision model trained on pothole imagery to detect and classify road defects in real time.

This manual covers everything needed to deploy, operate, retrain, and troubleshoot the system - from first-time SSH setup through full model retraining and production restart.

02 Prerequisites

2.1 Local Machine Requirements

These tools must be installed on your Mac or Linux workstation. They are used for syncing code and connecting to RunPod - they do not need to be on the server.

Tool	How to Install
Git	Pre-installed on most Macs. If missing, run: <code>brew install git</code>
Homebrew (Mac)	<code>/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"</code>
Add Homebrew to PATH	<code>eval "\$(/opt/homebrew/bin/brew shellenv zsh)"</code> — add to <code>~/.zprofile</code> to persist
rsync	<code>brew install rsync</code>
SSH key (<code>~/.ssh/id_ed25519</code>)	Must exist and be registered in RunPod (see Section 2.2)

2.2 Registering Your SSH Key with RunPod

Follow these steps once to enable password-free SSH access to your RunPod pods.

1 Copy your public key to the clipboard. e.g.

```
cat ~/.ssh/id_ed****.pub  
pbcopy < ~/.ssh/id_ed****.pub
```

2 Paste the key into RunPod

In RunPod, navigate to Settings → SSH Keys and update the key with the value you just copied.

3 Update your deploy script

Open `scripts/deploy_runpod.sh` and set the correct host and port for your pod: e.g.

```
RUNPOD_HOST="213.173.***.***"  
RUNPOD_PORT="22***"
```

 **Note:** Always use the "SSH over exposed TCP" option (`root@<ip> -p <port>`). The proxied option (`yf6p...@ssh.runpod.io`) does not support SCP/SFTP and will break rsync.

2.3 SSH Connection Reference

Replace the IP address and port number with the values shown for your active pod. e.g.

```
# Connect to RunPod
ssh root@213.173.***.*** -p 22**** -i ~/.ssh/id_ed25***
```

```
# Disconnect when done
exit
```

Warning: If SSH still prompts for a password after key setup, run the following inside the RunPod web terminal to set a fallback password:

```
wget https://raw.githubusercontent.com/justinwlin/Runpod-SSH-Password/main/passwordrunpod.sh \
&& chmod +x passwordrunpod.sh && ./passwordrunpod.sh
```

03 Running the App (Every Session)

Follow these four steps each time you want to use Pothole Eye. Steps 1 and 3 each require a separate terminal window on your Mac.

1 SSH into RunPod (Mac Terminal A) e.g.

```
ssh -i ~/.ssh/id_ed25*** -p ***** root@213.173.***.***
```

2 Start the application (RunPod Terminal) e.g.

```
cd /workspace/Pothole-I && bash scripts/start_app.sh
```

 **Note:** Your terminal prompt will show `root@<container-id>`, confirming you are on RunPod.

3 Open the SSH tunnel (Mac Terminal B — new window) e.g.

```
ssh -L 5001:localhost:5001 -i ~/.ssh/id_ed2**** -p 30*** root@213.173.***.**** -N
```

This command forwards the app port to your local machine and runs silently. Keep this terminal open for as long as you use the app.

4 Open the web interface

```
http://localhost:5001
```

Open this URL in any browser. The Pothole Eye interface should load within a few seconds.

3.1 Syncing Code Changes Before Starting

If you have made changes to the project on your Mac, sync them to RunPod before Step 2:

```
# Run on your Mac (before SSHing in)
bash scripts/deploy_runpod.sh
```

 **Note:** This script uses `rsync` to transfer only the files that have changed, keeping sync times short.

04 First-Time Deployment

Use this section the first time you set up the project on a new RunPod pod.

1 Sync project files from Mac to RunPod

```
cd /Users/<your-username>/Desktop/Pothole-I  
bash scripts/deploy_runpod.sh
```

This syncs all project files, then builds the GPU Docker image. On a first run, image build takes 5–15 minutes.

2 SSH into the pod

```
ssh -i ~/.ssh/id_ed25*** -p ***** root@213.173.***.***
```

3 Start the application

```
cd /workspace/Pothole-I && bash scripts/start_app.sh
```

Note: If you encounter locked package manager errors, run the following before `start_app.sh`:

```
kill -9 -f 'apt-get' 2>/dev/null; kill -9 -f 'dpkg' 2>/dev/null; sleep 2  
rm -f /var/lib/dpkg/lock-frontend /var/lib/dpkg/lock \  
/var/cache/apt/archives/lock /var/lib/apt/lists/lock  
dpkg --configure -a -q 2>/dev/null || true
```

05 Retraining the Model

Follow these steps to retrain the YOLOv8 model with an updated dataset. Training takes approximately 3–4 hours on an A40 GPU (48 GB VRAM).

1 Sync latest code from Mac to RunPod

```
cd /Users/<your-username>/Desktop/Pothole-I  
bash scripts/deploy_runpod.sh
```

2 SSH into the pod e.g.

```
ssh -i ~/.ssh/id_ed2*** -p ***** root@216.243.***.***
```

3 Navigate to the project directory

```
cd /workspace/Pothole-I
```

4 Confirm HuggingFace credentials are set

```
grep 'HF_' .env.gpu
```

Expected output:

```
HF_TOKEN=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  
HF_MODEL_REPO=mdmis001/pothole-eye-models
```

5 Check the dataset

```
ls /workspace/Pothole-I/Pothole-Detection-1/
```

If you see data.yaml, train/, and valid/ directories, skip to Step 6. If the directory is missing (fresh pod), download the dataset first:

```
source .env.gpu  
pip install roboflow --quiet  
python3 - <<'EOF'  
from roboflow import Roboflow  
rf = Roboflow(api_key='YOUR_ROBOFLOW_API_KEY')  
project = rf.workspace('your-workspace').project('pothole-detection')  
dataset = project.version(1).download('yolov8', location='/workspace/Pothole-I/Pothole-Detection-1')
```

```
EOF
```

6 Verify GPU availability

```
python3 scripts/verify_gpu.py
```

Expected: A40 with ~48 GB VRAM and CUDA available. Do not proceed if CUDA is not detected.

7 Start training

```
python3 training/train_gpu.py \  
  --model yolov8m.pt \  
  --epochs 150 \  
  --batch 16 \  
  --imgsz 1280 \  
  --name pothole_v2
```

Logs and metrics are written to runs/train/pothole_v2/. The --name flag prevents overwriting previous runs.

8 Monitor training (second SSH window)

```
# Open a second terminal and SSH in, then:  
cd /workspace/Pothole-I  
tail -f runs/train/pothole_v2/results.csv
```

Target metrics at epoch 150:

Metric	Target Value
mAP50	≥ 0.75
Precision	≥ 0.75
Recall	≥ 0.65

9 Verify training output

```
# Check weights are saved  
ls -lh /workspace/Pothole-I/models/  
  
# Check HuggingFace push succeeded  
python3 training/model_store.py status
```


Expect to see `pothole_yolov8.pt`, `best.pt`, and `last.pt` all recently modified. If the Hub push failed, run it manually:

```
python3 training/model_store.py push
```

10 Restart the app with the new model

Bare-metal start:

```
pkill -f 'python3 app.py' 2>/dev/null || true
pkill -f 'celery.*worker' 2>/dev/null || true
bash scripts/start_app.sh
```

Docker start:

```
docker compose -f docker-compose.gpu.yml restart pothole-detector celery-worker
```

11 Confirm the new model loaded

```
grep -E 'Pothole model|FOUND|pothole_yolov8|best\.pt' logs/app.log | tail -10
```

Look for a line similar to:

```
Pothole model ready. /workspace/Pothole-I/models/pothole_yolov8.pt
```

12 Sync weights back to Mac

```
# Run from your Mac terminal
bash scripts/deploy_runpod.sh
```

Step 5 of the deploy script automatically pulls the `models/` directory back to your Mac as a local backup.

06 HuggingFace Model Storage

Trained weights are automatically pushed to HuggingFace Hub at username/pothole-eye-models, keeping them safe between pod restarts. The following commands let you manage model storage manually.

6.1 Push Weights to Hub

```
cd /workspace/Pothole-I
python3 training/model_store.py push
```

This uploads models/pothole_yolov8.pt, models/last.pt, and metrics to the Hub repository.

6.2 Pull Weights on a New Pod

```
python training/model_store.py pull --dest models/
```

6.3 Installing Dependencies

RunPod's base image already includes the correct CUDA-built torch/torchvision wheels. To install all other project dependencies without downgrading PyTorch to a CPU-only version, use:

```
grep -vE '^(torch|torchvision|torchaudio|onnxruntime)' requirements.txt \
| grep -v '^#' | grep -v '^[[:space:]]*$' \
| pip install -r /dev/stdin --quiet
```

Warning: Do not run `pip install -r requirements.txt` directly - this will replace the GPU-optimised torch wheels with CPU-only versions.

07 Troubleshooting

If you encounter an error during training or deployment, consult the table below.

Error / Symptom	Recommended Fix
CUDA out of memory during training	Reduce batch size: add --batch 8 to the training command
Dataset not found (Step 5)	Re-run the dataset download script in Step 5
HuggingFace push failed: 403 Forbidden	Your token needs write scope. Regenerate it at huggingface.co/settings/tokens
App does not show the new model after training	Run Step 10 to restart the app. The model initializes once at startup and will not hot-reload.
SSH still asks for password after key setup	Run the passwordrunpod.sh script inside the RunPod web terminal (see Section 2.2)
rsync / SCP fails over proxied SSH	Switch to the direct TCP connection option (root@<ip> -p <port>). The proxied option does not support file transfer.
dpkg / apt locked on pod startup	Run the lock-cleanup commands shown in Section 4 before start_app.sh

08 Quick Reference — Common Commands

Start the App (every session) e.g.

```
# Terminal A - SSH in
ssh -i ~/.ssh/id_ed2**** -p ***** root@213.173.***.***

# RunPod terminal - start app
cd /workspace/Pothole-I && bash scripts/start_app.sh

# Terminal B - tunnel
ssh -L 5001:localhost:5001 -i ~/.ssh/id_ed2**** -p ***** root@213.173.***.***-N

# Browser
http://localhost:5001
```

Sync Code to Pod

```
bash scripts/deploy_runpod.sh
```

Train Model

```
python3 training/train_gpu.py --model yolov8m.pt --epochs 150 --batch 16 --imgsz
1280 --name pothole_v2
```

Push / Pull Model Weights

```
python3 training/model_store.py push
python3 training/model_store.py pull --dest models/
```

Restart App After Retraining

```
pkill -f 'python3 app.py' 2>/dev/null || true
bash scripts/start_app.sh
```